

Woxsen University

School of Technology

B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

ISIS

(Design and Implementation of a Database Management System for Health Insurance Policy and Claims Management System)

Course Code: ____25TU03MJM0____

Course Name: ____Database Management System____

Semester: ____III____

Academic Year: 2026–27

Submitted by:

Name: __Aaditi Pavankumar Chougule__

Register Number: __25WU0101002__

Section: ____CSE Core Tigers____

Name: __Anoushka Rajoria__

Register Number: __25WU0101014__

Section: ____CSE Core Tigers____

Name: __K.Hansika Reddy__

Register Number: __25WU0101060__

Section: ____CSE Core Tigers____

School of Technology

Department of Computer Science and Engineering

7th October 2026

CERTIFICATE

This is to certify that the Project titled “Design and Implementation of a Database Management System for Health Insurance Policy and Claims Management System” is a bonafide work carried out by the students of B.Tech – Computer Science and Engineering during the Academic Year 2026–27 under my supervision and guidance.

The work presented in this report describes the design, implementation, testing and evaluation of a relational database system for managing health insurance policies, claims, payments, assessments, settlements and appeals.

Guide Signature: _____

Name of the Guide: _____

Designation: _____

Date: _____

DECLARATION

I hereby declare that the project titled “Design and Implementation of a Database Management System for Health Insurance Policy and Claims Management System” submitted by us is an original academic work carried out under the guidance of the faculty mentioned in this report.

The project was developed as part of the Project Based Learning requirement. The database design, SQL implementation, sample data, queries, claim decision logic and testing presented in this report were developed for the purpose of demonstrating the concepts covered in the DBMS course.

All external sources used for the background and literature review have been acknowledged in the references section. The report has been prepared in accordance with the academic requirements provided for the project.

Date: ____07-10-2026____

Name: __Aaditi Pavankumar Chougule__

Register Number: __25WU0101002__

Name: __Anoushka Rajoria__

Register Number: __25WU0101014__

Name: __K.Hansika Reddy__

Register Number: __25WU0101060__

ACKNOWLEDGEMENT

We would like to thank Woxsen University and the School of Technology for providing us with the opportunity to work on this Project Based Learning project. The project helped us understand how the concepts taught in Database Management Systems can be applied to a practical problem.

We are grateful to our faculty guide for the guidance and feedback provided during the different stages of the project. The discussions about database design, relationships, normalization, SQL queries and implementation helped us improve the structure of our system.

We would also like to thank our teammates for their contribution to the database design, user interface, testing, documentation and presentation work. Working together helped us understand how different parts of a software project need to connect to form a complete system.

Finally, we thank everyone who supported us directly or indirectly during the development and testing of the Health Insurance Policy and Claims Management System.

ABSTRACT

Health insurance systems handle a large amount of related information such as customers, insurance plans, policies, premiums, payments, hospitals, treatments, claims, documents and settlements. Managing these records without a well-structured database can lead to repeated information, inconsistent records and difficulty in tracking the progress of a claim. This project presents a relational Database Management System for managing the main activities involved in health insurance policy and claims management.

The system was designed using an entity relationship model and implemented in MySQL. The database contains 16 normalized tables connected through primary and foreign keys. The design uses a separate associative table for the many-to-many relationship between insurance plans and treatments. Constraints such as UNIQUE, NOT NULL and CHECK are used to improve data integrity. Sample data was inserted and SQL queries were developed for policy, premium, claim, hospital, settlement and decision reports.

A major feature of the project is an Explainable Claim Decision System. A stored procedure named EvaluateClaim checks whether a policy is active, whether the treatment is covered, whether the treatment falls within the policy period, whether enough coverage remains, whether a possible duplicate claim exists and whether claim documents are verified. Based on these checks, the procedure recommends Approve, Reject or Manual Verification and also provides a risk level and explanation.

The project also includes database testing and a user interface connected to the database through an API layer. The final system demonstrates how relational database concepts, SQL queries, constraints, normalization, stored procedures and reporting can be combined to build a practical insurance management application.

TABLE OF CONTENTS

1. INTRODUCTION	9
2. PROBLEM STATEMENT AND OBJECTIVES	12
3. LITERATURE REVIEW / BACKGROUND	14
4. METHODOLOGY AND SYSTEM DESIGN	18
5. IMPLEMENTATION	24
6. RESULTS AND DISCUSSION	32
7. CONCLUSION AND FUTURE SCOPE	34
REFERENCES	35

LIST OF FIGURES

Figure 1. Overall system architecture

Figure 2. Claim evaluation flow

Figure 3. Data flow of the proposed system

Figure 4. ER diagram of the Health Insurance database

Figure 5. MySQL schema showing the implemented tables

Figure 6. Customer records in MySQL Workbench

Figure 7. Multi-table claim query result

Figure 8. Claim approval rate report

Figure 9. Explainable claim decision output

Figure 10. Hospital-wise claim report

LIST OF TABLES

Table 1. Project objectives

Table 2. Scope of the proposed system

Table 3. Main entities and their purpose

Table 4. Key relationships and cardinalities

Table 5. Functional dependencies

Table 6. Normalization summary

Table 7. Tools and technologies

Table 8. Database modules

Table 9. Important SQL operations

Table 10. Testing scenarios

Table 11. Results against objectives

1. INTRODUCTION

1.1 Background

Health insurance involves several types of information that are connected to each other. A customer may purchase a plan, receive a policy, pay premiums and later submit a claim for treatment received at a hospital. The claim may require documents, assessment, approval or rejection, settlement and sometimes an appeal. Because these activities are connected, storing everything in separate unlinked files can make it difficult to maintain consistency and retrieve complete information.

A relational database is suitable for this type of system because the information can be divided into entities and connected using keys. In our project, customer information is separated from plan information, policy information, treatment information and claim information. This reduces repeated data and makes it easier to maintain relationships. The database was implemented in MySQL using primary keys, foreign keys, unique constraints, check constraints and indexes.

The project contains 16 tables. Some tables represent main entities, while others represent transactions or relationships. For example, PlanTreatment resolves the many-to-many relationship between Plan and Treatment. ClaimDocument stores supporting documents, while ClaimDecision and DecisionCheck support the explainable decision feature.

The database is not intended to replace a complete commercial insurance platform. Instead, it is a practical academic implementation that demonstrates how DBMS concepts can be used to model an insurance workflow, enforce basic integrity rules, execute reports and support rule-based claim evaluation.

1.2 Motivation and Relevance to the Real-World Problem

The main motivation for this project was to take a familiar real-world process and understand how its information can be represented using database concepts. Insurance claim processing is a good example because one claim depends on several pieces of information. A claim cannot be evaluated only by looking at the claim amount. The system also needs to know the policy, the plan under which the policy was issued, the treatment, the hospital, the dates of treatment, the remaining coverage and the supporting documents.

Research on healthcare claims also shows that duplicate claims and other forms of incorrect or fraudulent activity are important problems. Recent reviews have studied data-driven and machine-learning approaches for detecting suspicious healthcare claims. These approaches can be powerful, but they normally require large datasets and more advanced analytics. Our project takes a smaller and more transparent approach. Instead of trying to predict fraud using a trained model, we use clearly defined database rules to identify conditions that need attention.

This choice is suitable for the scope of a DBMS course project. It allows the team to demonstrate SQL, relationships, normalization, constraints, joins, aggregation, subqueries, views and stored procedures while also adding a practical feature that goes beyond simple CRUD operations.

The system therefore connects two ideas: structured database management and explainable rule-based claim checking. The first provides reliable storage and retrieval, while the second demonstrates how business rules can be implemented using database logic.

1.3 Scope and Organization of the Report

The scope of the project covers the core data and database operations needed for a simplified health insurance policy and claims management system. Customer, plan, treatment, hospital and policy data are maintained as master information. Premiums and payments represent financial activity associated with policies. Claims, documents and assessments represent the claim-processing stage. Claim decisions, decision checks, settlements and appeals represent later stages of the workflow.

Area	Included in the project
Customer management	Customer and dependent information
Policy management	Plans, policies, validity dates and sum insured
Premium management	Premium dues and payment records
Treatment management	Treatments and plan-specific coverage
Claim management	Claim submission, documents and assessment
Decision support	Rule-based claim evaluation and explanations
Settlement	Settlement amount, status and payment reference
Appeal	Appeal reason, status and resolution
Reporting	Policy, premium, claim, hospital, settlement and decision reports

The report is organized according to the university template. The next chapter defines the problem and objectives. The literature review then discusses database design, healthcare data

systems, claim fraud detection and explainability. Methodology explains the architecture, ER model, relationships, normalization and workflow. Implementation describes the MySQL schema, sample data, queries, stored procedure and user interface integration. Results and discussion summarize testing and outputs, followed by the conclusion, future scope and references.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

The problem addressed in this project is the design of a database system that can store and manage the different types of information involved in health insurance policy and claims processing. In a typical process, information about a customer, policy, insurance plan, treatment, hospital, premium, claim and settlement is related. If the database structure is not designed carefully, the same information may be stored repeatedly and changes may not be reflected consistently.

A second problem is that a claim decision can be difficult to understand when only a final status such as Approved or Rejected is stored. For a practical system, it is useful to know whether the policy was active, whether the treatment was covered, whether enough coverage remained, whether a similar claim existed and whether documents were verified.

Therefore, our project has two connected goals. The first is to build a normalized relational database that maintains the required information and relationships. The second is to add a rule-based decision layer that produces an explainable claim recommendation.

2.2 Project Objectives

Objective	Explanation
Centralize insurance data	Maintain related insurance information in one structured MySQL database.
Reduce redundancy	Separate independent entities and use relationships instead of repeating the same information.
Maintain integrity	Use PK, FK, UNIQUE, NOT NULL and CHECK constraints.
Support reporting	Provide queries for policies, premiums, claims, settlements and hospitals.
Evaluate claims	Apply rule-based checks before recommending a claim decision.
Provide explanations	Store individual decision checks, severity and explanations.
Support UI integration	Allow the database to be accessed through a user interface and API layer.

2.3 Constraints and Assumptions

The project was developed with a number of practical constraints. First, the sample data is synthetic and is used only for demonstrating database operations. It does not represent real policyholders or real insurance claims. Second, the claim decision system is rule-based and is not a machine-learning fraud detection model. It is designed to demonstrate transparent business-rule processing within a DBMS.

The project assumes that policy dates, treatment dates, claim amounts and document verification status are entered correctly. It also assumes that the plan-treatment relationship contains the coverage percentage and treatment-specific limit needed for the decision procedure. The stored procedure uses processed settlements to estimate the remaining policy coverage.

Some business rules are better handled in a stored procedure or application layer rather than through a simple table CHECK constraint. For example, checking whether a treatment is covered requires looking at another table. Duplicate claim detection also depends on comparing multiple claim attributes. These cross-table rules are therefore implemented in the claim evaluation logic.

The user interface is treated as a client of the database through an API layer. The database remains the main source of truth. This is important because the project requirement states that changes made through the UI should be reflected in the MySQL database.

3. LITERATURE REVIEW / BACKGROUND

3.1 Database Systems for Structured Information

Database design literature emphasizes that a good relational database begins with identifying entities, attributes, keys and relationships. Silberschatz, Korth and Sudarshan explain the importance of relational modeling, integrity constraints, transaction concepts and query processing in database systems [1]. Elmasri and Navathe similarly describe conceptual design using ER models and normalization as a way of converting a real-world information system into a structured relational schema [2].

These ideas directly influenced our design. We started by identifying the main objects involved in health insurance, such as Customer, Plan, Policy, Treatment, Hospital and Claim. We then separated transaction and supporting information into related tables. The final structure uses foreign keys to connect these tables and avoids placing unrelated attributes in one large table.

Coronel and Morris describe database design as an iterative process in which business requirements are translated into entities, relationships and constraints [3]. This was useful for our project because the claim workflow contains several stages. The database had to represent not only the claim itself but also documents, assessment, decision, settlement and appeal information.

3.2 MySQL Constraints, Queries and Database Objects

The MySQL Reference Manual documents the use of CREATE TABLE, primary keys, unique constraints, foreign keys and CHECK constraints for defining relational structures [4], [5]. Foreign keys are especially important because they allow related records to remain consistent across parent and child tables. Our project uses foreign keys between customers and policies, policies and claims, claims and documents, and several other relationships.

Aggregate functions such as COUNT, SUM, AVG and MAX are used to summarize sets of rows and are commonly combined with GROUP BY [6]. We use these functions in reports such as claims by status, hospital-wise claims, settlement summaries and approval rate.

MySQL also supports stored procedures as stored database objects that can be invoked using CALL [7]. This feature was used for our EvaluateClaim procedure. Views were used to create reusable report definitions for active policies and pending claims [8].

The important point from these technical references is that database systems provide more than tables. Constraints protect data, queries retrieve and summarize information, views simplify repeated reporting, and stored procedures allow business logic to be placed close to the data. Our project combines these features in one application.

3.3 Healthcare Claims and Fraud Detection Research

Healthcare claims have also been studied from the perspective of fraud and anomaly detection. A systematic review by du Preez and colleagues examined 137 studies on machine-learning methods for healthcare claim fraud detection and reported the use of supervised, unsupervised and hybrid approaches [9]. The review also highlights problems such as inconsistent data, limited labeled fraudulent cases, privacy concerns and data standardization.

A separate systematic review of healthcare fraud detection found that duplicate claims and claims for services that were not actually provided are among the types of suspicious activity studied in the literature [10]. These findings are relevant to our project because duplicate claim detection is one of the rules included in our decision procedure.

A scoping review of health insurance fraud between 2006 and 2020 also identifies multiple forms of fraud involving providers, beneficiaries and insurers [11]. The range of possible fraud patterns shows why a production system would need more advanced analytics than our academic prototype.

Our approach is intentionally narrower. We do not claim to detect all fraud. Instead, we detect a small set of conditions that are directly understandable from the relational data already available in the system. This makes the feature practical for a DBMS project and keeps the decision process transparent.

3.4 Explainability and Healthcare Decision Support

Explainability has become an important topic in healthcare technology because decisions that affect patients and financial outcomes need to be understandable. Holzinger and colleagues discuss causability and explainability in medicine and highlight the importance of making decision processes understandable, especially when complex models are used [12]. Tjoa and Guan review explainable AI for medical applications and point out the importance of interpretability in domains where decisions require accountability [13].

Markus, Kors and Rijnbeek discuss the role of explainability in creating trustworthy AI for healthcare and note that transparency is an important barrier to clinical adoption [14]. A systematic scoping review of explainable AI using electronic health record data also shows that different explanation approaches are used, including intrinsically interpretable models and rule extraction [15].

Our project is not an XAI or machine-learning system. Instead, we use a directly interpretable rule-based approach. Each rule is visible in the stored procedure and each check can be stored in DecisionCheck with PASS, FAIL or WARNING, severity and an explanation. This provides a simple form of traceability without introducing an opaque model.

The literature therefore supports the motivation for transparency, while our implementation keeps the solution within the scope of a relational database project.

3.5 Comparison of Related Approaches

Related work / approach	Main focus	Limitation for our project	What we used
Relational DBMS textbooks [1], [2]	ER modeling, keys, normalization	General concepts, not insurance-specific	ER model, relational schema, 3NF
MySQL documentation [4]-[8]	SQL structures and database objects	Technical reference rather than application design	Tables, FK, CHECK, joins, aggregates, views, procedure
Healthcare fraud review [9]	ML-based fraud detection	Needs data, training and validation	Used as background; no ML
Healthcare fraud review [10]	Fraud patterns and detection methods	Broad healthcare focus	Duplicate-claim rule
Fraud scoping review [11]	Types and factors of insurance fraud	Wider than our prototype	Used to motivate validation
Explainability research [12]-[15]	Transparent decision support	Mostly focused on AI/XAI	Rule-based explanations
EHR prediction research	Large-scale predictive	Complex and data intensive	Not implemented; future

[16]	healthcare models		direction
------	-------------------	--	-----------

The comparison shows the position of our project. It is primarily a DBMS implementation, not a fraud analytics or machine-learning system. The contribution is the combination of a normalized insurance database with an explainable, rule-based claim decision layer. The approach is simple enough to understand, test and demonstrate in MySQL while still addressing a practical problem in claim processing.

4. METHODOLOGY AND SYSTEM DESIGN

4.1 Overall Architecture

The system follows a layered structure. The user interface provides screens for viewing and modifying insurance information. An API layer receives requests from the interface and communicates with MySQL. The MySQL database contains the 16 normalized tables, reporting views and the EvaluateClaim stored procedure. Query results are returned to the interface and can also be inspected directly in MySQL Workbench.

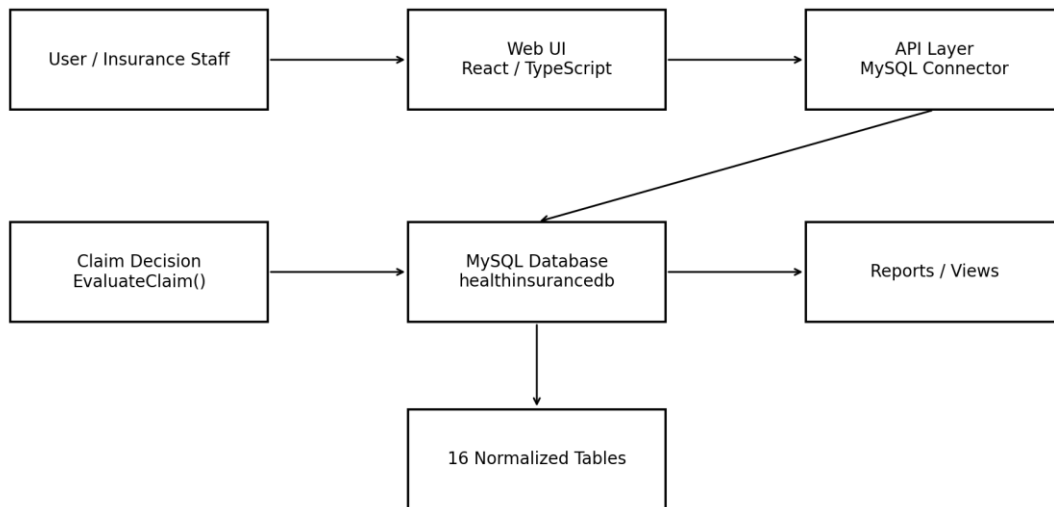


Figure 1. Overall system architecture of the proposed Health Insurance Policy and Claims Management System.

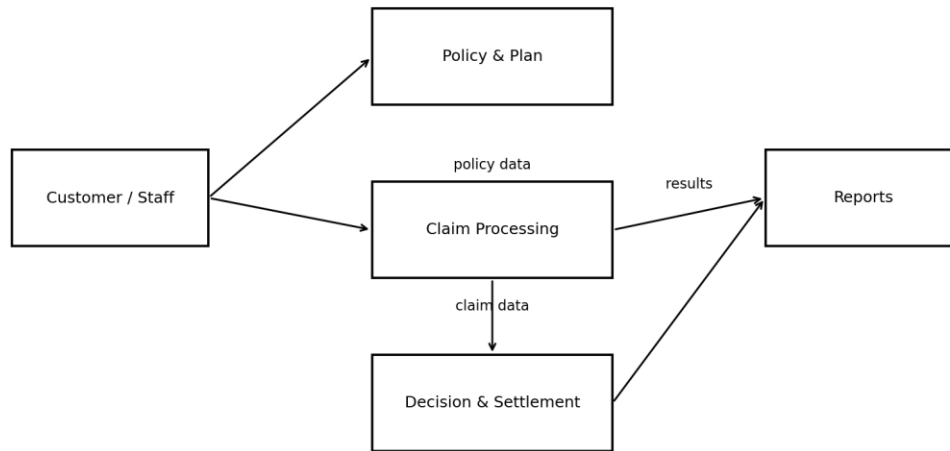


Figure 3. High-level data flow between users, policy/claim processing, decision logic and reports.

This separation makes the database independent from the visual interface. The same database can be accessed through SQL queries, the API or another client application. It also makes testing easier because database operations can be checked directly in MySQL.

4.2 Entity and Relationship Design

The conceptual design was created by identifying the objects required for the insurance workflow. Customer stores policyholder information. Plan stores insurance products. Treatment stores medical treatments. Policy connects a customer to a plan. Hospital represents the treatment location. Claim connects a policy, hospital and treatment. The remaining tables represent supporting information around premiums, documents, assessments, decisions, settlements and appeals.

Entity	Purpose	Main key
Customer	Policyholder details	customer_id PK
Plan	Insurance plan details	plan_id PK
Treatment	Medical treatments	treatment_id PK
PlanTreatment	Plan-treatment coverage	(plan_id, treatment_id) PK
Policy	Customer-plan contract	policy_id PK
Dependent	People covered under policy	dependent_id PK
Premium	Premium dues	premium_id PK
Payment	Premium payments	payment_id PK
Hospital	Hospital details	hospital_id PK

Claim	Insurance claim	claim_id PK
ClaimDocument	Claim documents	document_id PK
Assessment	Claim assessment	assessment_id PK
ClaimDecision	Overall decision	decision_id PK
DecisionCheck	Individual checks	check_id PK
Settlement	Processed settlement	settlement_id PK
Appeal	Appeal records	appeal_id PK

4.3 ER Diagram and Cardinality

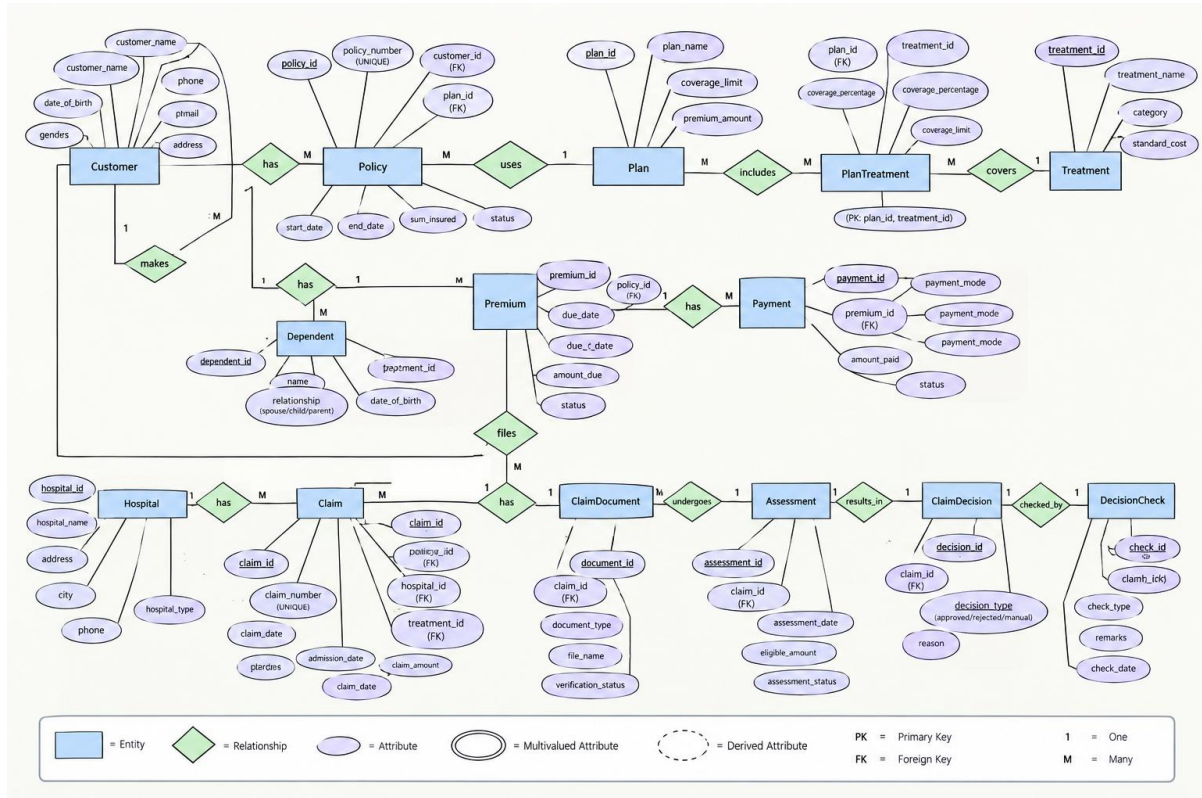


Figure 4. ER diagram showing the main entities, keys and relationships in the project.

The ER diagram shows the central role of Policy and Claim. Customer and Plan connect to Policy. Plan and Treatment are connected through PlanTreatment. Policy connects to Dependents, Premiums and Claims. Claim connects to Hospital and Treatment and then expands into documents, assessment, decision, settlement and appeal information.

The important cardinalities are Customer 1:M Policy, Plan 1:M Policy, Plan M:N Treatment through PlanTreatment, Policy 1:M Dependent, Policy 1:M Premium, Premium 1:M Payment, Policy 1:M Claim, Hospital 1:M Claim, Treatment 1:M Claim, Claim 1:M ClaimDocument, Claim 1:1 Assessment, Claim 1:1 ClaimDecision, ClaimDecision 1:M DecisionCheck, Claim 1:1 Settlement and Claim 1:M Appeal.

4.4 Relational Schema and Keys

The relational schema translates the ER model into tables. Each table has a primary key. Foreign keys connect related tables. Some business identifiers are also marked UNIQUE. For example, `policy_number` and `claim_number` are unique because they represent external identifiers that should not be duplicated.

The remaining tables follow the same pattern. The relational schema was kept normalized so that customer details, plan details, treatment details and hospital details do not need to be copied into transaction tables.

4.5 Functional Dependencies and Normalization

Functional dependencies were used to check whether attributes are placed in the correct tables. For example, `customer_id` determines the customer name, date of birth, gender, phone, email and address. Similarly, `policy_id` determines the policy number, `customer_id`, `plan_id`, dates, sum insured and status.

Table	Functional dependency
Customer	<code>customer_id</code> $\rightarrow$ <code>customer_name</code> , <code>date_of_birth</code> , <code>gender</code> , <code>phone</code> , <code>email</code> , <code>address</code>
Plan	<code>plan_id</code> $\rightarrow$ <code>plan_name</code> , <code>description</code> , <code>coverage_limit</code> , <code>premium_amount</code> , <code>plan_status</code>
Treatment	<code>treatment_id</code> $\rightarrow$ <code>treatment_name</code> , <code>category</code> , <code>description</code> , <code>standard_cost</code>
Policy	<code>policy_id</code> $\rightarrow$ <code>policy_number</code> , <code>customer_id</code> , <code>plan_id</code> , <code>dates</code> , <code>sum_insured</code> , <code>status</code>
Claim	<code>claim_id</code> $\rightarrow$ <code>claim_number</code> , <code>policy_id</code> , <code>hospital_id</code> , <code>treatment_id</code> , <code>dates</code> , <code>amount</code> , <code>status</code>
PlanTreatment	<code>(plan_id, treatment_id)</code> $\rightarrow$ <code>coverage_percentage</code> , <code>coverage_limit</code>

The PlanTreatment table is the key example for partial dependency analysis. Neither `plan_id` nor `treatment_id` alone determines the coverage values because coverage depends on the specific combination of a plan and treatment. Therefore the composite key correctly represents the dependency.

4.6 Normalization up to Third Normal Form

Normal form	Application in our project
1NF	Attributes are atomic; repeating and multi-valued information is separated into related tables.
2NF	Non-key attributes depend on the complete key. PlanTreatment has no partial dependency on only one part of its composite key.
3NF	Independent concepts are stored separately, so non-key attributes do not depend on other non-key attributes.

In the first normal form, fields such as phone, email, address, policy dates and claim amount contain individual values. Multiple documents are not stored in a single Claim field; instead, ClaimDocument allows multiple document records.

In the second normal form, tables with single-column keys automatically avoid partial dependency. The important composite-key table, PlanTreatment, stores only values that depend on both `plan_id` and `treatment_id`.

In the third normal form, customer attributes are kept in Customer rather than in Policy or Claim. Plan attributes are kept in Plan, treatment attributes in Treatment and hospital

attributes in Hospital. This prevents changes to a single concept from requiring updates in many unrelated records.

5. IMPLEMENTATION

5.1 Database Creation and Table Implementation

The database was implemented in MySQL using InnoDB tables. The structure script first creates the database and then creates the tables in an order that respects the foreign key dependencies. Primary keys use `AUTO_INCREMENT` where appropriate. Foreign keys establish the relationships between parent and child tables.

```
CREATE DATABASE HealthInsuranceDB;
USE HealthInsuranceDB;

CREATE TABLE Customer (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  ...
);

CREATE TABLE Policy (
  policy_id INT AUTO_INCREMENT PRIMARY KEY,
  policy_number VARCHAR(20) NOT NULL UNIQUE,
  customer_id INT NOT NULL,
  plan_id INT NOT NULL,
  ...
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
  FOREIGN KEY (plan_id) REFERENCES Plan(plan_id)
);
```

The design also uses `CHECK` constraints for valid values. Examples include positive monetary amounts, coverage percentages between 0 and 100, valid plan status values and valid hospital types. These constraints provide a first layer of validation at the database level.

5.2 Sample Data and Data Population

After the structure was created, sample records were inserted into the database. The data was designed to test normal and exceptional situations rather than only successful cases. The sample includes customers, plans, treatments, hospitals, policies, dependants, premiums, payments, claims, documents, assessments, decisions, decision checks, settlements and appeals.

Test scenario	Example purpose
Active policy + covered treatment	Normal approval path
Expired policy	Tests rejection based on policy validity
Possible duplicate claim	Tests Manual Verification path
High-value claim	Tests coverage limit logic
Missing/unverified documents	Tests document warning logic
Processed settlement	Tests remaining coverage calculation

The sample claims were deliberately selected to produce different decision outcomes. Claim 1 represents a normal approved case. Claim 2 uses an expired policy and is rejected. Claim 3 is similar to an existing claim and is sent for manual verification. Other claims demonstrate covered treatments and treatment-specific reimbursement limits.

5.3 SQL Queries and Reports

The query script demonstrates the main SQL operations required by the project. Basic SELECT statements retrieve complete tables, while more detailed queries combine several tables using JOIN. WHERE filters active policies or pending claims. ORDER BY sorts results. GROUP BY is used with COUNT, SUM and AVG for reports.

```
SELECT
  cl.claim_number,
  c.customer_name,
  p.policy_number,
  h.hospital_name,
  t.treatment_name,
  cl.claim_amount,
  cl.status
FROM
  Claim cl
JOIN Policy p ON cl.policy_id = p.policy_id
JOIN Customer c ON p.customer_id = c.customer_id
JOIN Hospital h ON cl.hospital_id = h.hospital_id
JOIN Treatment t ON cl.treatment_id = t.treatment_id;
```

This query is useful because a user usually wants a complete claim view rather than only the numeric foreign keys. The result combines the claim number, customer, policy, hospital, treatment, amount and status.

Other reports include premium dues, premium payment summaries, claims by status, hospital-wise claims, settlement summaries, approval rate, decision explanations, failed or warning checks, duplicate claims, claims above the average amount, plan-wise summaries, appeals and document verification.

5.4 Aggregate, Subquery and View Implementation

Aggregate queries provide a summary instead of returning every individual record. For example, claims can be grouped by status and summarized using COUNT, SUM and AVG. A hospital-wise report uses LEFT JOIN so that hospitals can still appear even if they do not currently have claims.

```

SELECT
  status,
  COUNT(*)
  SUM(claim_amount)
  AVG(claim_amount)
FROM
GROUP
ORDER BY number_of_claims DESC;
AS
AS
AS
BY
number_of_claims,
total_claim_amount,
average_claim_amount
Claim
status

```

The project also uses a subquery to identify claims whose amount is above the average claim amount. Another query uses the same average to identify customers with high-value claims. A self-join is used for duplicate claim detection by comparing the Claim table with itself.

Two views, ActivePolicyReport and PendingClaimReport, were created for repeated reporting. A view provides a convenient way to access a stored SELECT definition without rewriting the full query each time.

5.5 Explainable Claim Decision System

The main novelty of the project is the Explainable Claim Decision System. It is implemented as a MySQL stored procedure named EvaluateClaim. The procedure receives a claim ID and retrieves the claim, policy and plan information required for evaluation.

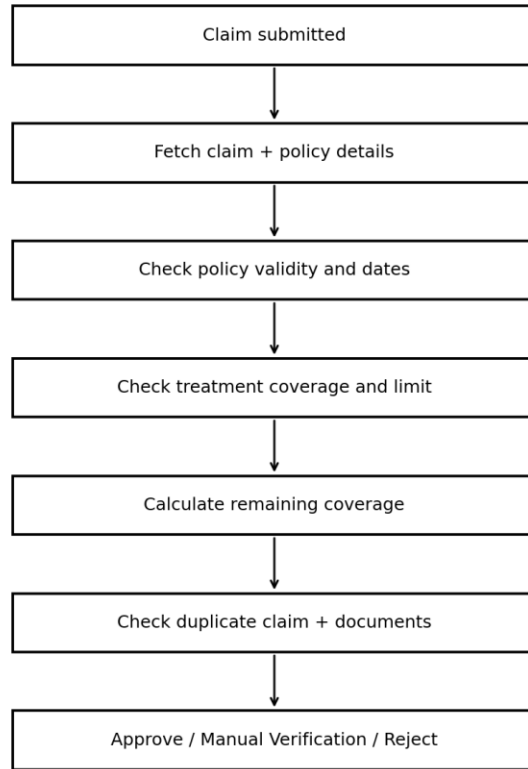


Figure 2. Claim evaluation flow implemented by the EvaluateClaim stored procedure.

The procedure first checks whether the claim exists. It then retrieves policy status, start date, end date, sum insured and plan ID. The PlanTreatment table is checked to determine whether the treatment is covered and what percentage and treatment-specific limit apply. The eligible amount is calculated using the lower of the percentage-based amount and treatment limit.

The procedure also calculates remaining policy coverage by subtracting processed settlements for the same policy. Duplicate detection compares policy, hospital, treatment and admission/discharge dates. Document verification counts submitted and unverified documents.

5.6 Decision Rules and Explainability

The decision logic follows a fixed priority order. An invalid or expired policy results in Reject with High risk. If the treatment is not covered, the claim is also rejected. If the remaining policy coverage is less than the eligible reimbursement amount, the claim is rejected. A possible duplicate claim results in Manual Verification with High risk. Missing or

unverified documents result in Manual Verification with Medium risk. If all required checks pass, the recommendation is Approve with Low risk.

Condition	Decision	Risk
Policy inactive or treatment outside policy period	Reject	High
Treatment not covered	Reject	High
Insufficient remaining coverage	Reject	High
Possible duplicate claim	Manual Verification	High
Missing or unverified documents	Manual Verification	Medium
All required checks pass	Approve	Low

The procedure returns the individual check results using CASE expressions. It also returns the number of similar claims, the number of unverified documents, the recommended decision, risk level and a text explanation. This makes the result easier to understand than a single status field.

5.7 Database Testing

Testing was performed using a separate SQL testing script. The purpose was to check both normal retrieval and important business rules. Basic tests verify that records exist in the required tables and that joins return the expected relationships. Additional tests check decisions, settlements, appeals, approval rate, duplicate claims and stored procedure outputs.

Test	Expected outcome
Retrieve Customer records	Customer rows are displayed
Retrieve Claim records	Claim rows are displayed
Claim-to-policy join	Correct policy information appears for each claim
Treatment coverage query	Only covered plan-treatment combinations are returned
Document verification	Verified and unverified documents can be identified
Approval rate	Approved, rejected and manual counts are calculated
Duplicate detection	Matching claim pairs are identified
EvaluateClaim(1)	Normal approved scenario
EvaluateClaim(2)	Expired policy rejected
EvaluateClaim(3)	Possible duplicate sent to manual verification
Views	Active and pending reports return data

The testing also checks the integrity constraints defined in the database. Invalid values such as negative amounts or invalid relationships should be rejected by the appropriate constraints. Cross-table rules are tested through the stored procedure.

5.8 User Interface and Database Connectivity

The user interface was developed as a separate client layer. Its purpose is to make database operations easier for a user who does not need to write SQL for every action. The interface communicates with an API layer, which communicates with the MySQL database.

The main database-related operations required for the project are viewing records, inserting records and deleting records. The interface also provides claim evaluation so that the stored procedure can be executed from the application. The database remains the source of truth, which means that a successful UI operation should be visible when the corresponding table is queried in MySQL.

The project source code includes a MySQL connection configuration and API routes for insurance-related operations. The final setup uses environment variables for database connection details so that credentials do not need to be hard-coded into the source code.

For demonstration, the UI can be used to add a record, verify that the record appears in MySQL, delete the test record and verify the deletion. A claim can then be evaluated and the returned decision and explanation can be displayed to the user.

5.9 Implementation Screenshots and Outputs

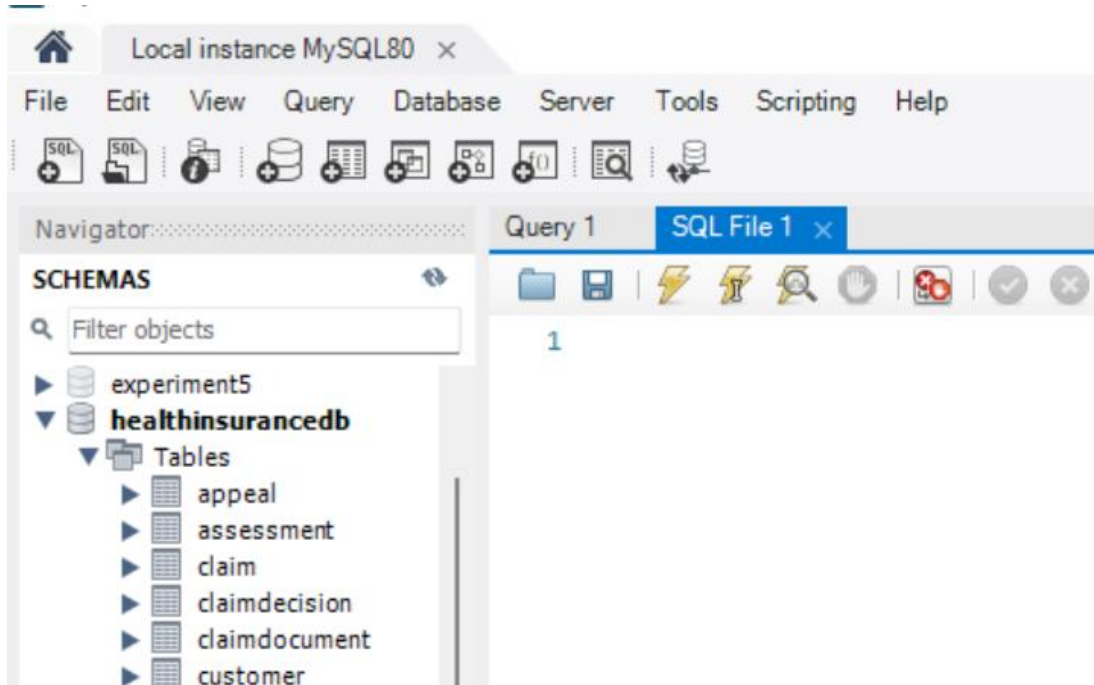


Figure 5. MySQL Workbench showing the healthinsurancedb schema and implemented tables.

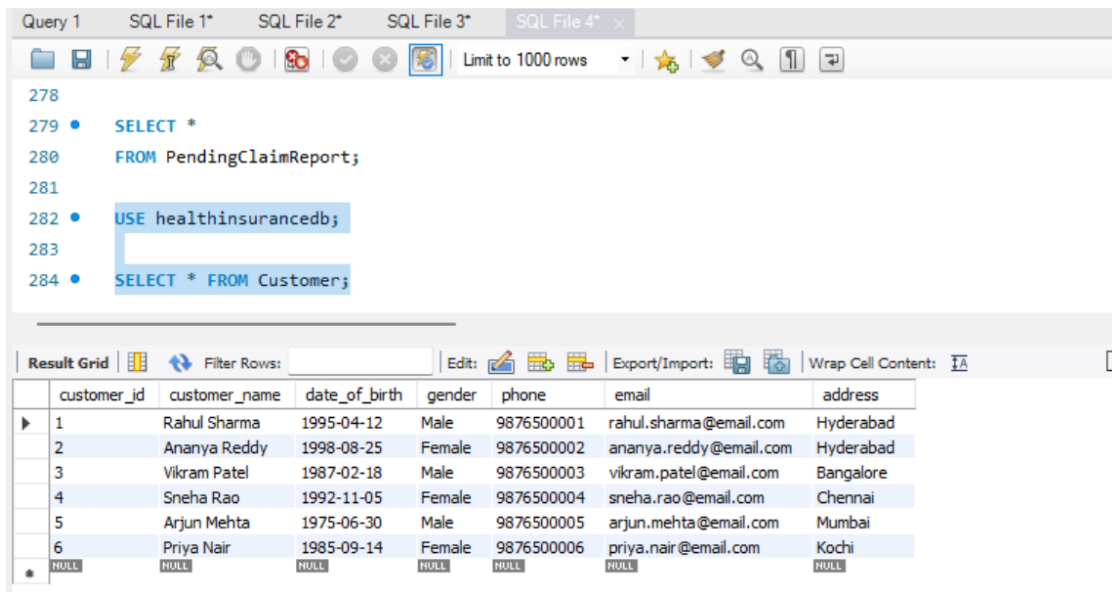


Figure 6. Customer records retrieved from the implemented database.

The screenshots confirm that the database was created successfully and populated with sample records. The schema view shows the tables used in the project, while the customer query demonstrates retrieval from the database.

5.10 Query and Decision Outputs

```

299 JOIN Customer c
300     ON p.customer_id = c.customer_id
301 JOIN Hospital h
302     ON cl.hospital_id = h.hospital_id
303 JOIN Treatment t
304     ON cl.treatment_id = t.treatment_id;

```

	claim_number	customer_name	policy_number	hospital_name	treatment_name	claim_amount	status
▶	CLM1001	Rahul Sharma	POL1001	Apollo Hospitals	Appendectomy	80000.00	Settled
	CLM1003	Rahul Sharma	POL1001	Apollo Hospitals	Appendectomy	80000.00	Manual Verification
	CLM1004	Ananya Reddy	POL1002	KIMS Hospitals	Knee Replacement	450000.00	Rejected
	CLM1002	Vikram Patel	POL1003	Yashoda Hospitals	Appendectomy	75000.00	Rejected
	CLM1006	Sneha Rao	POL1004	Sunshine Hospitals	Pneumonia Hospitalization	70000.00	Settled
	CLM1005	Arjun Mehta	POL1005	Fortis Hospital	Knee Replacement	280000.00	Settled
	CLM1007	Arjun Mehta	POL1005	Yashoda Hospitals	Cataract Surgery	60000.00	Settled
	CLM1008	Priya Nair	POL1006	KIMS Hospitals	Fracture Treatment	50000.00	Settled

Figure 7. Multi-table claim query combining claim, customer, policy, hospital and treatment information.

```

307
308 SELECT

```

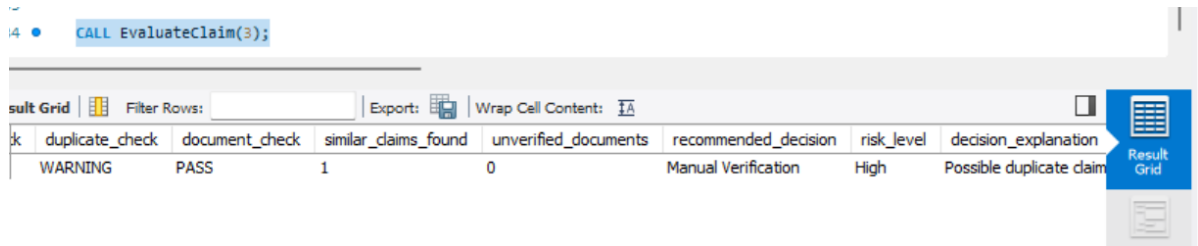
	total_decisions	approved	rejected	manual_verification	approval_percentage
▶	8	5	2	1	62.50

Figure 8. Claim approval rate report using aggregate functions.

The multi-table result demonstrates the practical use of JOIN. Instead of showing only foreign-key values, it presents a readable claim report. The approval-rate result demonstrates COUNT, conditional SUM and ROUND. These outputs were used to verify that the database supports the required reporting operations.

6. RESULTS AND DISCUSSION

6.1 Explainable Decision Result



```
CALL EvaluateClaim(3);
```

duplicate_check	document_check	similar_claims_found	unverified_documents	recommended_decision	risk_level	decision_explanation
WARNING	PASS	1	0	Manual Verification	High	Possible duplicate claim

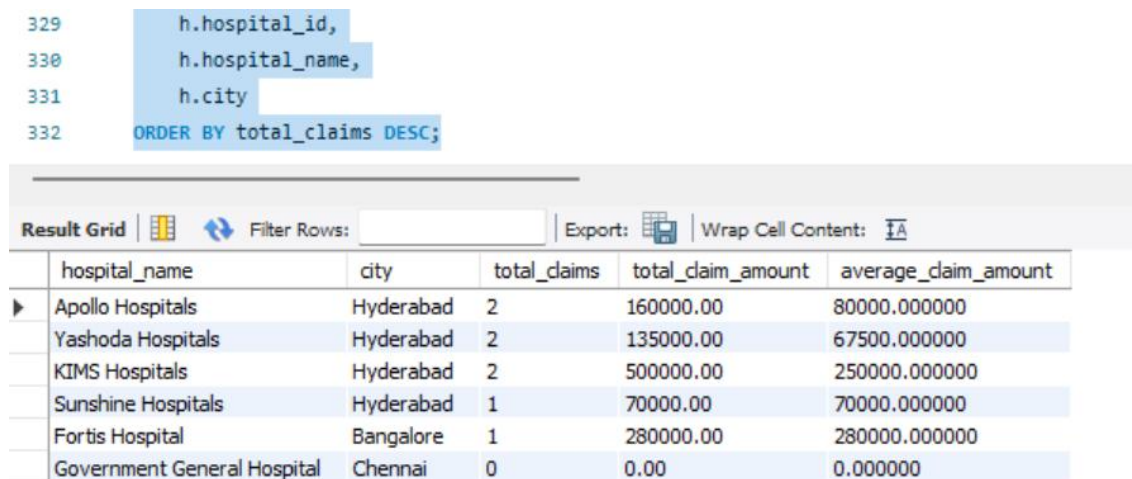
Figure 9. EvaluateClaim output showing individual checks and the recommended decision.

The most important result is the output of EvaluateClaim. For the duplicate-claim test, the duplicate check is reported as a warning, a similar claim is found, the documents are verified, and the recommended decision is Manual Verification with High risk. This is useful because the system does not hide the reason behind the recommendation.

The result also demonstrates the difference between an automatic rejection and a case that requires human attention. A possible duplicate is not automatically treated as a confirmed fraud case. Instead, it is flagged for manual verification. This is a safer approach for an academic prototype because the system only identifies a condition that requires review.

6.2 Hospital and Settlement Reporting

```
329     h.hospital_id,  
330     h.hospital_name,  
331     h.city  
332 ORDER BY total_claims DESC;
```



hospital_name	city	total_claims	total_claim_amount	average_claim_amount
Apollo Hospitals	Hyderabad	2	160000.00	80000.000000
Yashoda Hospitals	Hyderabad	2	135000.00	67500.000000
KIMS Hospitals	Hyderabad	2	500000.00	250000.000000
Sunshine Hospitals	Hyderabad	1	70000.00	70000.000000
Fortis Hospital	Bangalore	1	280000.00	280000.000000
Government General Hospital	Chennai	0	0.00	0.000000

Figure 10. Hospital-wise claim report using LEFT JOIN and aggregate functions.

The hospital report demonstrates that the database can be used for management-level summaries. It groups claims by hospital and city and calculates the number of claims, total claim amount and average claim amount. Similar reports are available for settlements, premium dues, claim status, plans, appeals and document verification.

The settlement report links settlement records back to claims and customers. This is possible because the database maintains the relationship between Settlement, Claim, Policy and Customer. The design therefore supports both transaction-level information and summary-level reporting.

The reporting queries also help validate the schema. If foreign keys or relationships were incorrect, multi-table reports would either fail or return incomplete results. Successful report generation therefore provides an additional practical check of the database design.

6.3 Results Against Objectives

Objective	Result
Centralized insurance data	16 related tables implemented in one MySQL database.
Reduce redundancy	Independent entities were separated and normalized to 3NF.
Maintain integrity	PK, FK, UNIQUE, NOT NULL and CHECK constraints were implemented.
Support reporting	JOIN, aggregate, subquery and view-based reports were implemented.
Evaluate claims	EvaluateClaim procedure checks policy, treatment, coverage, duplicates and documents.
Provide explanations	DecisionCheck and procedure output provide check-level explanations.
Support UI integration	API-based connection prepared for viewing, insertion, deletion and claim evaluation.

Overall, the project meets the main DBMS objectives defined at the beginning. The database is structured, connected and testable. The decision system adds a practical business-rule layer without changing the relational foundation. The results also show that the database can support both detailed records and summarized reports.

7. CONCLUSION AND FUTURE SCOPE

The Health Insurance Policy and Claims Management System demonstrates how a real-world information process can be converted into a normalized relational database. The project includes 16 tables representing customers, plans, treatments, policies, dependants, premiums, payments, hospitals, claims, documents, assessments, decisions, decision checks, settlements and appeals.

The database uses primary and foreign keys to establish relationships and constraints to protect data quality. The design was normalized to 3NF to reduce redundancy and keep independent information in the correct tables. SQL queries demonstrate joins, filtering, grouping, aggregate functions, subqueries and views.

The main project contribution is the Explainable Claim Decision System. The EvaluateClaim procedure applies transparent rules and returns a decision, risk level and explanation. The use of DecisionCheck makes the decision traceable and allows a reviewer to see which checks passed, failed or produced warnings.

The project also provided practical learning in database design, SQL implementation, testing and application integration. Future development can make the system closer to a production insurance platform.

- Add role-based authentication for policy administrators, claim assessors and customers.
- Add transaction management for multi-step claim and settlement operations.
- Add audit logs to track every change made to policy and claim records.
- Connect the system to secure document storage and automated document verification.
- Extend duplicate and anomaly detection using statistical or machine-learning methods.
- Add dashboards and trend analysis for claim volumes, settlement amounts and hospital activity.
- Introduce stronger privacy and security controls for sensitive health and personal information.

REFERENCES

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [3] C. Coronel and S. Morris, Database Systems: Design, Implementation, & Management, 13th ed. Cengage, 2018.
- [4] Oracle, “MySQL 8.4 Reference Manual: CREATE TABLE Statement,” MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/create-table.html>
- [5] Oracle, “MySQL 8.4 Reference Manual: FOREIGN KEY Constraints,” MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>
- [6] Oracle, “MySQL 8.4 Reference Manual: Aggregate Functions,” MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html>
- [7] Oracle, “MySQL 8.4 Reference Manual: Stored Objects,” MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/stored-objects.html>
- [8] Oracle, “MySQL 8.4 Reference Manual: Views FAQ,” MySQL Documentation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/faqs-views.html>
- [9] A. du Preez, S. Bhattacharya, P. Beling, and E. Bowen, “Fraud detection in healthcare claims using machine learning: A systematic review,” Artificial Intelligence in Medicine, vol. 160, 103061, 2025, doi: 10.1016/j.artmed.2024.103061.
- [10] “How to detect healthcare fraud? A systematic review,” Gaceta Sanitaria, vol. 35, suppl. 2, pp. S441–S449, 2021, doi: 10.1016/j.gaceta.2021.07.022.

- [11] “Fourteen years of manifestations and factors of health insurance fraud, 2006–2020: A scoping review,” *BMC Health Services Research*, 2021.
- [12] A. Holzinger, G. Langs, H. Denk, K. Zatloukal, and H. Müller, “Causability and explainability of artificial intelligence in medicine,” *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 4, e1312, 2019, doi: 10.1002/widm.1312.
- [13] E. Tjoa and C. Guan, “A survey on explainable artificial intelligence (XAI): Towards medical XAI,” *IEEE Transactions on Neural Networks and Learning Systems*, 2019.
- [14] A. F. Markus, J. A. Kors, and P. R. Rijnbeek, “The role of explainability in creating trustworthy artificial intelligence for health care: A comprehensive survey,” *Journal of Biomedical Informatics*, vol. 113, 103655, 2021.
- [15] A. J. Amann et al., “Explainable artificial intelligence models using real-world electronic health record data: A systematic scoping review,” *Journal of the American Medical Informatics Association*, 2020, doi: 10.1093/jamia/ocaa053.
- [16] A. Rajkomar, E. Oren, K. Chen et al., “Scalable and accurate deep learning with electronic health records,” *npj Digital Medicine*, vol. 1, no. 18, 2018, doi: 10.1038/s41746-018-0029-1.
- [17] A. Rajkomar, J. Dean, and I. Kohane, “Machine learning in medicine,” *New England Journal of Medicine*, vol. 380, no. 14, pp. 1347–1358, 2019, doi: 10.1056/NEJMr1814259.
- [18] N. S. Al-Ansary, “Artificial Intelligence in Health Insurance Fraud Detection: A Global and Regional Systematic Review,” *Frontiers in Public Health*, vol. 14, 2026, doi: 10.3389/fpubh.2026.1863242.